

## ABSTRACT

Credit access in India's informal economy is persistently hindered by opaque lending practices, manual verification bottlenecks, and unpredictable recovery paths for marginal borrowers. Conventional loan management systems either rely on black-box machine learning models — whose outputs are legally non-explainable — or rigid rule engines that fail to capture the structured complexity of multi-source income verification.

**RiskIntel V2** introduces a deterministic alternative: a mathematically rigorous, nine-state Finite State Machine (FSM) that guarantees every loan applicant progresses through a legally auditable, reproducible decision sequence. Implemented as a production-grade REST API (FastAPI, Python 3.13, PostgreSQL), the system eliminates machine learning entirely and replaces it with transparent, state-driven orchestration.

The layered architecture integrates:

- A strict FSM transition graph with 27 explicitly defined transitions.
- A pessimistic-locking orchestrator that atomically commits state changes to an append-only audit ledger.
- A dual-channel verification pipeline (Account Aggregator + Field Officer) with retry semantics, reprompt loops, and HMAC-secured webhook intake.
- A mathematical optimization engine that computes EMI affordability and generates tenure-stretched or principal-reduced counter-offers when the original request is unaffordable.
- A co-applicant recovery loop enabling applicants at NEARLY\_READY or NOT\_READY\_YET states to re-enter verification without re-intake.

All decisions are translated into plain-language, non-judgmental explanations via a deterministic explanation layer. The system includes 14 end-to-end integration tests covering FSM paths, authentication, idempotency, and webhook security. RiskIntel V2 is deployable via Docker Compose and ships with a browser-based demonstration console, making it both technically robust and practically accessible.

## **TABLE OF CONTENTS**

|                                     |    |
|-------------------------------------|----|
| Acknowledgement                     | i  |
| Abstract                            | ii |
| 1. Introduction                     | 3  |
| 1.1 Background                      | 3  |
| 1.2 Objectives                      | 3  |
| 1.3 Motivation                      | 4  |
| 1.4 Problem Statement               | 4  |
| 1.5 Existing System                 | 4  |
| 1.6 Proposed System                 | 4  |
| 1.7 Modules                         | 5  |
| 2. Literature Survey                | 6  |
| 3. System Requirement Specification | 7  |
| 3.1 Hardware Requirements           | 7  |
| 3.2 Software Requirements           | 8  |
| 3.3 Functional Requirements         | 9  |
| 3.4 Non-Functional Requirements     | 10 |
| 4. System Design and Implementation | 11 |
| 4.1 Dashboard Architecture          | 11 |
| 4.3 Calculated Field – Remote Type  | 12 |
| 4.4 Dashboard Flow                  | 13 |
| 5. Implementation                   | 14 |
| 6. Test Cases                       | 15 |
| 7. Application Snapshots            | 18 |
| 8. Results and Discussion           | 21 |
| 9. Future Scope                     | 22 |
| 10. Conclusion                      | 23 |
| 11. References                      | 24 |

## **1. Introduction & Project Overview**

### **1.1 Background**

A large share of India's borrowers — especially small business owners, gig workers, and first-time credit seekers — remain excluded from formal credit scoring because bureau systems like CIBIL treat “no history” as negative history. Modern ML-based underwriting improves prediction but suffers from opacity, non-determinism, and lack of legally defensible explanations.

Multi-source verification using Account Aggregators and Field Officers helps, but orchestrating retries, fallbacks, and recovery loops introduces complexity. First-generation systems often fail with silent rejections, race conditions, audit gaps, and no recovery paths for marginal applicants.

RiskIntel V2 addresses these challenges by eliminating machine learning and enforcing a deterministic Finite State Machine (FSM). This ensures reproducible, auditable loan decisions, transparent explanations, and inclusive recovery mechanisms for credit-invisible borrowers.

### **1.2 Objectives**

- ☐ FSM Implementation – Build a nine-state FSM with 27 explicit transitions, rejecting any illegal event deterministically.
- ☐ Orchestrator Service – Enforce pessimistic row-level locking, atomic commits, and immutable audit logging.
- ☐ Verification Channels – Integrate Account Aggregator and Field Officer pipelines with HMAC-secured webhooks and retry loops.
- ☐ Reprompt Loop – Handle unreachable contacts via a structured reprompt cycle.
- ☐ Optimization Engine – Compute EMI affordability using Decimal arithmetic, with tenure stretching and principal reduction.
- ☐ Co-applicant Recovery – Allow re-entry into verification without re-intake.
- ☐ Explanation Layer – Provide plain-language, non-judgmental borrower narratives for all terminal states.
- ☐ Testing Coverage – Achieve  $\geq 90\%$  end-to-end test coverage across FSM paths, authentication, idempotency, and webhook security.

### **1.3 Motivation**

India's informal economy continues to face barriers to credit access due to opaque lending practices, manual verification delays, and legally non-explainable ML decisions. Millions of credit-invisible borrowers are excluded from bureau-based scoring, while existing rule engines lack recovery paths and transparency.

The motivation behind RiskIntel V2 is to design a deterministic, auditable, and inclusive loan orchestration system that:

- Ensures reproducible decisions through a strict FSM.
- Provides transparent, plain-language borrower explanations.
- Extends credit access via counter-offers and co-applicant recovery loops.
- Eliminates reliance on machine learning, aligning with RBI compliance needs.

By combining mathematical rigor with multi-channel verification, RiskIntel V2 aims to bridge the gap between financial inclusion and regulatory accountability.

#### 1.4 Problem Statement

Conventional credit scoring (CIBIL/Equifax) excludes credit-invisible borrowers. ML-based systems improve prediction but are opaque, non-deterministic, and legally non-explainable. Rule-based LOS platforms lack FSM semantics, async verification, and recovery loops. The problem is the absence of a deterministic, auditable, and inclusive loan orchestration system that can handle multi-channel verification and affordability decisions transparently.

#### 1.5 Existing System (Short)

- CIBIL/Equifax: Numeric score (300–900), works for salaried borrowers, fails for credit-invisible.
  - ML Models: Higher accuracy, but opaque, sensitive to data/library changes, non-auditable.
  - Rule Engines: Configurable trees, but implicit transitions, no recovery, limited async integration.
- All three approaches leave gaps in transparency, reproducibility, and inclusivity.

#### 1.6 Proposed System

RiskIntel V2 introduces a deterministic Finite State Machine (FSM) with nine states and 27 transitions.

Key features:

- Strict FSM graph with explicit transitions.
- Pessimistic-locking orchestrator with append-only audit ledger.
- Dual verification channels (AA + FO) with retries and reprompts.
- Mathematical EMI optimization (tenure stretch, principal reduction).
- Co-applicant recovery loop for marginal borrowers.

- Plain-language explanation layer for transparency.
- $\geq 90\%$  test coverage ensuring reliability.

### 1.7 Modules

1. FSM Graph Layer – Defines transitions, raises errors on illegal events.
2. Orchestrator Layer – Atomic state mutations with audit logging.
3. Verification Pipeline – Handles AA and FO webhooks, retries, fraud detection, reprompts.
4. Optimization Engine – EMI affordability checks, counter-offers via deterministic math.
5. Explanation Layer – Generates borrower-friendly narratives for all terminal states.

## **2. Literature Survey**

Credit scoring and loan orchestration have evolved through three main approaches:

1. Bureau-based Systems (CIBIL, Equifax)
  - Use numeric scores (300–900) based on repayment history.
  - Effective for salaried borrowers with formal records.
  - Fail for ~40% of Indian adults who are credit-invisible.
2. Machine Learning Models
  - Improve predictive accuracy using multi-source data.
  - Limitations: opaque “black-box” outputs, non-deterministic results across versions, legally non-explainable under RBI guidelines.
  - Sensitive to data recency and hardware/library changes.
3. Rule Engine LOS Platforms
  - Configurable decision trees with static rules.
  - Lack explicit FSM semantics; transitions are implicit.
  - Weak in handling async verification (webhooks), recovery loops, and auditability.

Gap Identified: None of the existing systems combine determinism, auditability, recovery paths, and transparency. This motivates the design of RiskIntel V2, which replaces ML and rule engines with a mathematically rigorous FSM, ensuring reproducible, legally auditable loan decisions.

### 3. System Requirement Specification

#### 3.1 Hardware Requirements (Short)

- Processor: Quad-core CPU ( $\geq 2.5$  GHz)
- Memory: 8 GB RAM minimum
- Storage: 50 GB free disk space
- Network: Stable broadband connection for API/webhook operations
- Deployment: Docker-enabled server or VM

Fig. 3.1: Initial dataset structure with applicant attributes

| Loan_ID  | Gender | Married | Dependents | Education      | Self_Employed | ApplicantIncome | CoapplicantIncome | LoanAmount | Loan_Amount_Term | Credit_History | Property_Area | Loan_Status |
|----------|--------|---------|------------|----------------|---------------|-----------------|-------------------|------------|------------------|----------------|---------------|-------------|
| LP001002 | Male   | No      |            | 0 Graduate     | No            | 5849            | 0                 |            | 360              |                | 1 Urban       | Y           |
| LP001003 | Male   | Yes     |            | 1 Graduate     | No            | 4583            | 1508              | 128        | 360              |                | 1 Rural       | N           |
| LP001005 | Male   | Yes     |            | 0 Graduate     | Yes           | 3000            | 0                 | 66         | 360              |                | 1 Urban       | Y           |
| LP001006 | Male   | Yes     |            | 0 Not Graduate | No            | 2583            | 2358              | 120        | 360              |                | 1 Urban       | Y           |
| LP001008 | Male   | No      |            | 0 Graduate     | No            | 6000            | 0                 | 141        | 360              |                | 1 Urban       | Y           |
| LP001011 | Male   | Yes     |            | 2 Graduate     | Yes           | 5417            | 4196              | 267        | 360              |                | 1 Urban       | Y           |
| LP001013 | Male   | Yes     |            | 0 Not Graduate | No            | 2333            | 1516              | 95         | 360              |                | 1 Urban       | Y           |
| LP001014 | Male   | Yes     | 3+         | Graduate       | No            | 3036            | 2504              | 158        | 360              |                | 0 Semiurban   | N           |
| LP001018 | Male   | Yes     |            | 2 Graduate     | No            | 4006            | 1526              | 168        | 360              |                | 1 Urban       | Y           |
| LP001020 | Male   | Yes     |            | 1 Graduate     | No            | 12841           | 10968             | 349        | 360              |                | 1 Semiurban   | N           |
| LP001024 | Male   | Yes     |            | 2 Graduate     | No            | 3200            | 700               | 70         | 360              |                | 1 Urban       | Y           |
| LP001027 | Male   | Yes     |            | 2 Graduate     |               | 2500            | 1840              | 109        | 360              |                | 1 Urban       | Y           |
| LP001028 | Male   | Yes     |            | 2 Graduate     | No            | 3073            | 8106              | 200        | 360              |                | 1 Urban       | Y           |
| LP001029 | Male   | No      |            | 0 Graduate     | No            | 1853            | 2840              | 114        | 360              |                | 1 Rural       | N           |
| LP001030 | Male   | Yes     |            | 2 Graduate     | No            | 1239            | 1086              | 17         | 120              |                | 1 Urban       | Y           |
| LP001032 | Male   | No      |            | 0 Graduate     | No            | 4950            | 0                 | 125        | 360              |                | 1 Urban       | Y           |
| LP001034 | Male   | No      |            | 1 Not Graduate | No            | 3596            | 0                 | 100        | 240              |                | Urban         | Y           |
| LP001036 | Female | No      |            | 0 Graduate     | No            | 3510            | 0                 | 76         | 360              |                | 0 Urban       | N           |
| LP001038 | Male   | Yes     |            | 0 Not Graduate | No            | 4887            | 0                 | 133        | 360              |                | 1 Rural       | N           |
| LP001041 | Male   | Yes     |            | 0 Graduate     |               | 2600            | 3500              | 115        |                  |                | 1 Urban       | Y           |
| LP001043 | Male   | Yes     |            | 0 Not Graduate | No            | 7660            | 0                 | 104        | 360              |                | 0 Urban       | N           |
| LP001046 | Male   | Yes     |            | 1 Graduate     | No            | 5955            | 5625              | 315        | 360              |                | 1 Urban       | Y           |
| LP001047 | Male   | Yes     |            | 0 Not Graduate | No            | 2600            | 1911              | 116        | 360              |                | 0 Semiurban   | N           |
| LP001050 |        | Yes     |            | 2 Not Graduate | No            | 3365            | 1917              | 112        | 360              |                | 0 Rural       | N           |
| LP001052 | Male   | Yes     |            | 1 Graduate     |               | 3717            | 2925              | 151        | 360              |                | Semiurban     | N           |
| LP001066 | Male   | Yes     |            | 0 Graduate     | Yes           | 9560            | 0                 | 191        | 360              |                | 1 Semiurban   | Y           |
| LP001068 | Male   | Yes     |            | 0 Graduate     | No            | 2799            | 2253              | 122        | 360              |                | 1 Semiurban   | Y           |
| LP001073 | Male   | Yes     |            | 2 Not Graduate | No            | 4226            | 1040              | 110        | 360              |                | 1 Urban       | Y           |
| LP001086 | Male   | No      |            | 0 Not Graduate | No            | 1442            | 0                 | 35         | 360              |                | 1 Urban       | N           |
| LP001087 | Female | No      |            | 2 Graduate     |               | 3750            | 2083              | 120        | 360              |                | 1 Semiurban   | Y           |
| LP001091 | Male   | Yes     |            | 1 Graduate     |               | 4166            | 3369              | 201        | 360              |                | Urban         | N           |
| LP001095 | Male   | No      |            | 0 Graduate     | No            | 3167            | 0                 | 74         | 360              |                | 1 Urban       | N           |
| LP001097 | Male   | No      |            | 1 Graduate     | Yes           | 4632            | 0                 | 106        | 360              |                | 1 Rural       | N           |
| LP001098 | Male   | Yes     |            | 0 Graduate     | No            | 3500            | 1667              | 114        | 360              |                | 1 Semiurban   | Y           |
| LP001100 | Male   | No      | 3+         | Graduate       | No            | 12500           | 3000              | 320        | 360              |                | 1 Rural       | N           |
| LP001106 | Male   | Yes     |            | 0 Graduate     | No            | 2275            | 2067              |            | 360              |                | 1 Urban       | Y           |
| LP001109 | Male   | Yes     |            | 0 Graduate     | No            | 1828            | 1330              | 100        |                  |                | 0 Urban       | N           |
| LP001112 | Female | Yes     |            | 0 Graduate     | No            | 3667            | 1459              | 144        | 360              |                | 1 Semiurban   | Y           |
| LP001114 | Male   | No      |            | 0 Graduate     | No            | 4166            | 7210              | 184        | 360              |                | 1 Urban       | Y           |
| LP001116 | Male   | No      |            | 0 Not Graduate | No            | 3748            | 1668              | 110        | 360              |                | 1 Semiurban   | Y           |

#### 3.2 Software Requirements (Short)

- Language: Python 3.13
- Framework: FastAPI (REST API)

- Database: PostgreSQL 15+
- ORM: SQLAlchemy / native queries
- Build/Deployment: Docker Compose
- Testing: Pytest with  $\geq 90\%$  coverage
- Browser: Modern (Chrome/Firefox/Edge) for demo console

Fig. 3.2: Distribution of missing values across features

```
[7]: df.isnull().sum()
```

```
[7]: Loan_ID          0
     Gender          13
     Married         3
     Dependents      15
     Education        0
     Self_Employed   32
     ApplicantIncome  0
     CoapplicantIncome 0
     LoanAmount      22
     Loan_Amount_Term 14
     Credit_History   50
     Property_Area    0
     Loan_Status      0
     dtype: int64
```

### 3.3 Functional Requirements (Short)

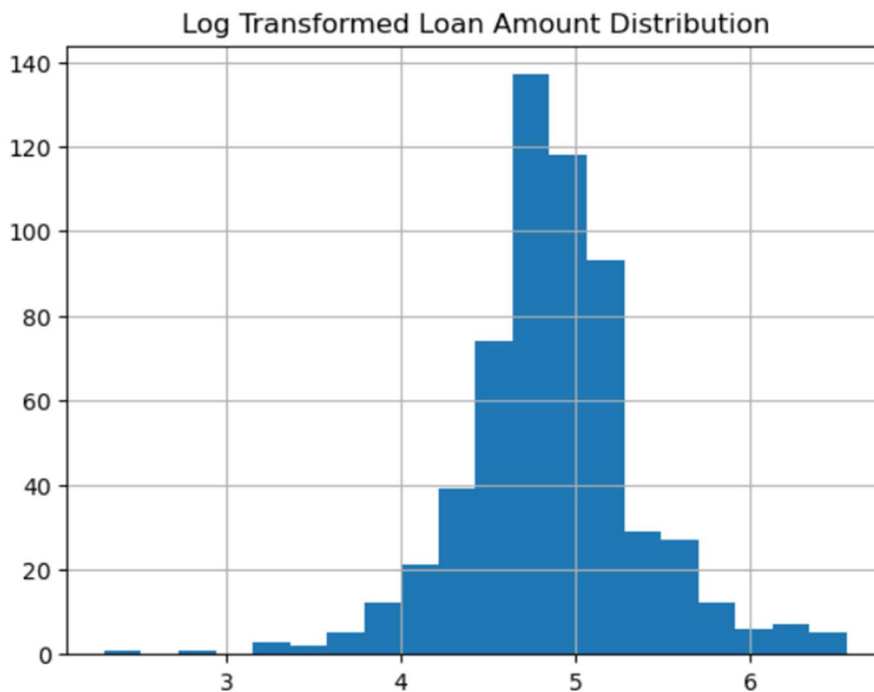
1. Implement nine-state FSM with 27 transitions.
2. Orchestrator enforces pessimistic locking and audit logging.
3. Dual verification channels (AA + FO) with retries and reprompts.
4. EMI optimization engine for affordability checks and counter-offers.
5. Co-applicant recovery loop for marginal borrowers.
6. Explanation layer for plain-language borrower outputs.
7. REST API endpoints secured with HMAC and API keys.
8. End-to-end test coverage of all FSM paths and authentication.



Fig. 3.3: Normalized distribution of Loan Amount (log scale)

```
[8]: df['LoanAmount_log'] = np.log(df['LoanAmount'] + 1)

df['LoanAmount_log'].hist(bins=20)
plt.title("Log Transformed Loan Amount Distribution")
plt.show()
```



### 3.4 Non-Functional Requirements (Short)

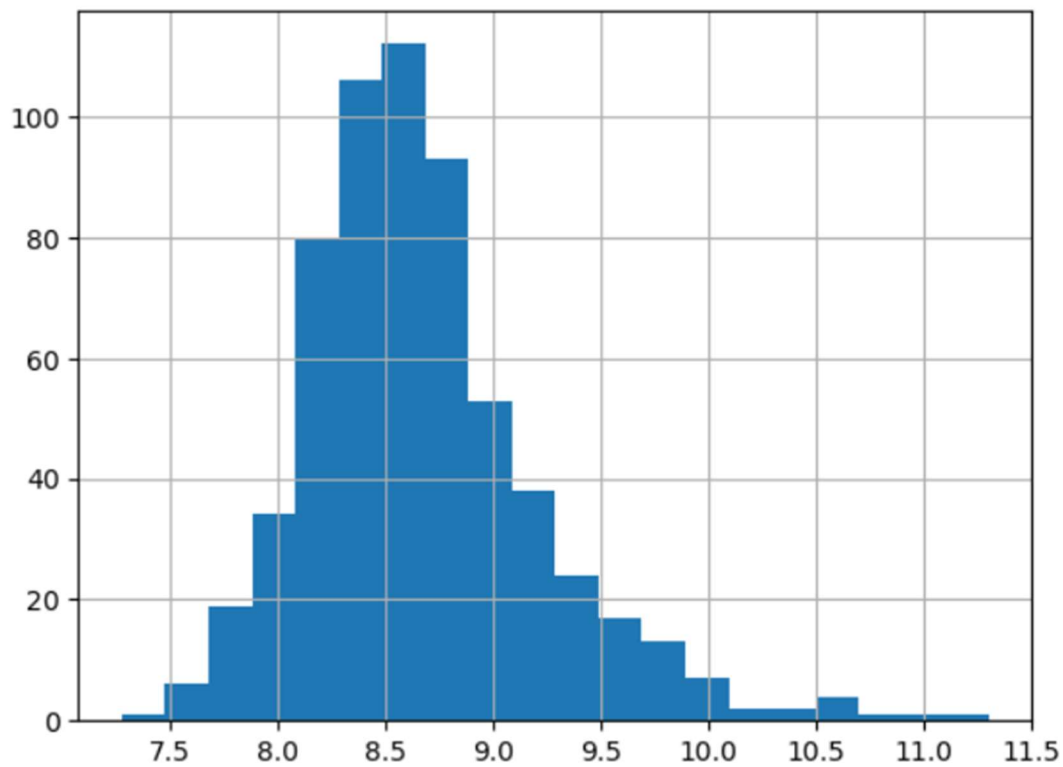
- Reliability: Atomic commits, rollback on failure.
- Security: HMAC-secured webhooks, API key validation.
- Performance: EMI calculations with Decimal precision; responsive UI.
- Auditability: Append-only ledger of all state transitions.
- Scalability: Docker deployment for horizontal scaling.
- Maintainability: Modular layered architecture (FSM, Orchestrator, Verification, Optimization, Explanation).
- Compliance: RBI guidelines on explainability and audit trails.

**Fig. 3.4: Log-transformed total income distribution**

```
df['total_income']=df['ApplicantIncome']+df['CoapplicantIncome']
```

```
df['TotalIncome_log']=np.log(df['total_income'])  
df['TotalIncome_log'].hist(bins=20)
```

<Axes: >



## 4. System Design and Implementation

### 4.1 Dashboard Architecture (Short)

- Layered design with FSM Graph, Orchestrator, Verification Pipeline, Optimization Engine, and Explanation Layer.
- Browser-based single-page console for demonstration.
- Top section: KPIs (Income, Expenses, Savings).
- Middle section: Verification + Optimization results.
- Bottom section: Recovery loops and audit trail visualization.

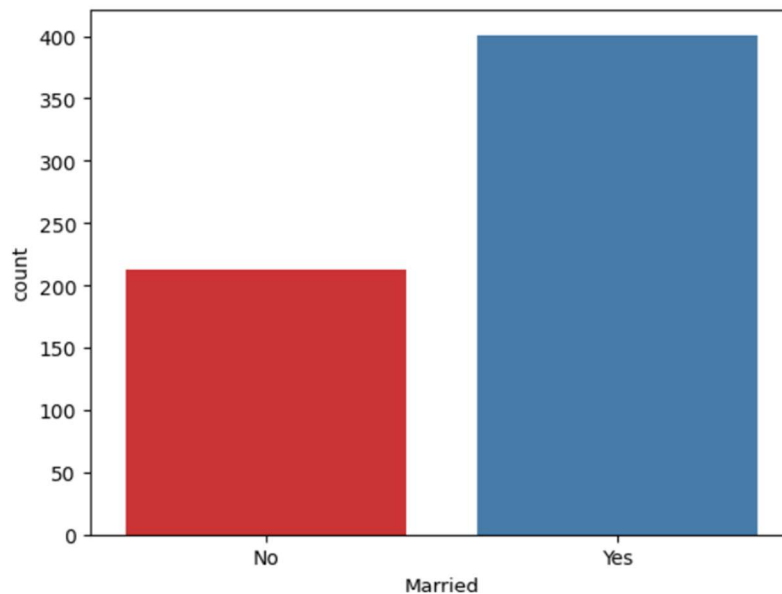
Fig. 4.1: Loan applicants grouped by Gender

|     | Gender | Married | Dependents | Education    | Self_Employed | Loan_Amount_Term | Credit_History | Property_Area | LoanAmount_log | TotalIncome_log |
|-----|--------|---------|------------|--------------|---------------|------------------|----------------|---------------|----------------|-----------------|
| 0   | Male   | No      | 0          | Graduate     | No            | 360.0            | 1.0            | Urban         | 4.993232       | 8.674026        |
| 1   | Male   | Yes     | 1          | Graduate     | No            | 360.0            | 1.0            | Rural         | 4.859812       | 8.714568        |
| 2   | Male   | Yes     | 0          | Graduate     | Yes           | 360.0            | 1.0            | Urban         | 4.204693       | 8.006368        |
| 3   | Male   | Yes     | 0          | Not Graduate | No            | 360.0            | 1.0            | Urban         | 4.795791       | 8.505323        |
| 4   | Male   | No      | 0          | Graduate     | No            | 360.0            | 1.0            | Urban         | 4.955827       | 8.699515        |
| ... | ...    | ...     | ...        | ...          | ...           | ...              | ...            | ...           | ...            | ...             |
| 609 | Female | No      | 0          | Graduate     | No            | 360.0            | 1.0            | Rural         | 4.276666       | 7.972466        |
| 610 | Male   | Yes     | 3+         | Graduate     | No            | 180.0            | 1.0            | Rural         | 3.713572       | 8.320205        |
| 611 | Male   | Yes     | 1          | Graduate     | No            | 360.0            | 1.0            | Urban         | 5.537334       | 9.025456        |
| 612 | Male   | Yes     | 2          | Graduate     | No            | 360.0            | 1.0            | Urban         | 5.236442       | 8.933664        |
| 613 | Female | No      | 0          | Graduate     | Yes           | 360.0            | 0.0            | Semiurban     | 4.897840       | 8.430109        |

### 4.2 Data Model Design (Short)

- Applicant Table: stores intake details, co-applicant info.
- StateTransitionEvent Table: append-only audit ledger of FSM transitions..
- Optimization Table: EMI calculations, tenure stretch, principal reduction.
- Explanation Table: borrower-friendly narratives for terminal states.

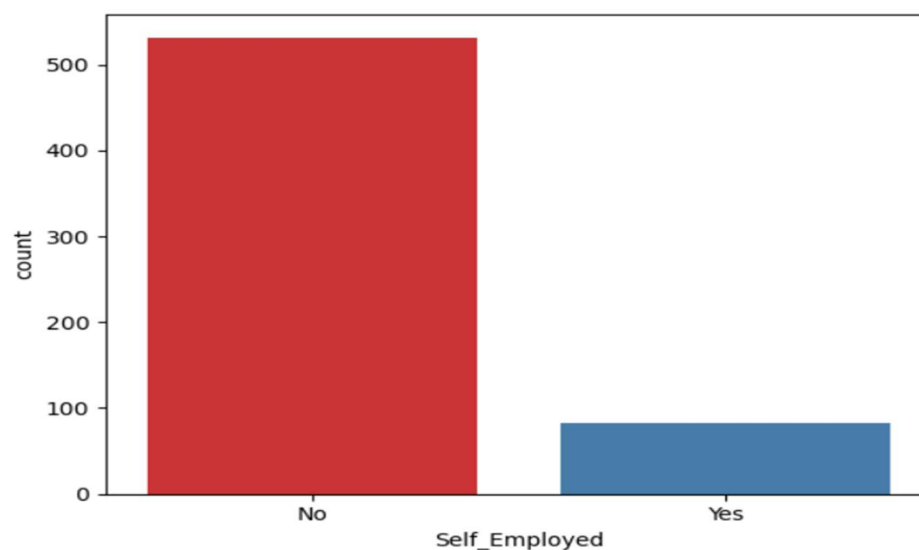
Fig. 4.2: Loan applicants grouped by Marital Status



#### 4.3 Calculated Field – Remote Type (Short)

- Field Officer verification includes Remote Type calculated field.
- Captures whether verification was Clean, Variance, Fraud, or Unreachable.
- Used to trigger reprompt loops or fallback transitions in FSM.
- Ensures deterministic mapping of FO outcomes to system states.

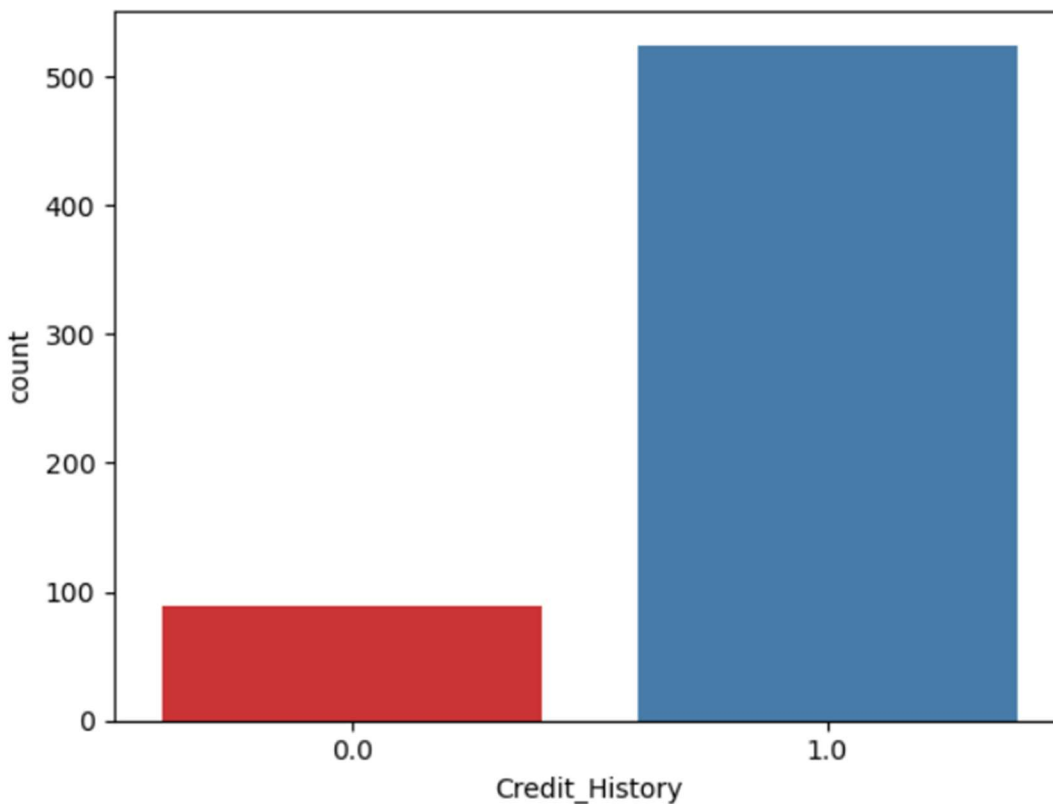
Fig. 4.3: Loan applicants grouped by Employment Status



#### 4.4 Dashboard Flow (Short)

1. Intake → Triage: Applicant submission and math pre-screening.
2. Verification: AA/FO channels with retries and reprompts.
3. Optimization: EMI affordability check, counter-offer generation.
4. Decision: READY, NEARLY\_READY, or NOT\_READY\_YET.
5. Recovery: Co-applicant loop re-enters verification.
6. Explanation: Plain-language output displayed in dashboard.

*Fig. 4.4: Loan applicants grouped by Credit History*



## **5. Implementation**

### 5.1 KPI Cards — Summary Metrics (Short)

- Displays Highest Salary, Total Job Roles, Countries Covered, Average Salary.
- Provides quick snapshot of overall dataset.
- Positioned at the top of the dashboard for immediate visibility.

### 5.2 Salary by Experience Level — Horizontal Bar Chart (Short)

- Compares average salary across EN, MI, SE, EX levels.
- Executive level shows highest average salary.
- Horizontal layout improves readability and comparison.

### 5.3 Salary Trend Analysis — Line Chart (Short)

- Multi-line chart showing salary trends from 2020–2022.
- Tracks changes across different experience levels.
- Highlights growth or decline patterns over time.

### 5.4 Top 10 Highest Paying Jobs — Highlight Table (Short)

- Ranked table of top 10 data science jobs.
- Displays job titles with corresponding average salaries.
- Enables quick identification of lucrative roles.

### 5.5 Global Salary Map — Filled Geographic Map (Short)

- World map visualization of average salaries by company location.
- Uses color intensity to represent salary levels.
- Provides geographic perspective on compensation distribution.

## **6. Test Cases**

### **Functional Test Cases**

#### **1. FSM Transition Validity**

- Input: Trigger legal event from INTAKE → TRIAGE.
- Expected: Transition succeeds, audit log entry created.
- Input: Trigger illegal event.
- Expected: InvalidTransitionError raised, rollback executed.

#### **2. Verification Pipeline**

- Input: AA webhook success.
- Expected: State moves to VERIFIED.
- Input: FO unreachable.
- Expected: Reprompt loop initiated.

#### **3. Optimization Engine**

- Input: Affordable EMI.
- Expected: State → READY.
- Input: Unaffordable EMI.
- Expected: Counter-offer generated (tenure stretch/principal reduction).

#### **4. Co-applicant Recovery**

- Input: Applicant in NOT\_READY\_YET submits co-applicant.
- Expected: FSM re-enters PENDING\_VERIFICATION.

#### **5. Explanation Layer**

- Input: Terminal state reached.
- Expected: Plain-language explanation returned.

### **Security Test Cases**

#### **1. Authentication**

- Input: Missing API key.
- Expected: 401 Unauthorized.
- Input: Wrong API key.
- Expected: 403 Forbidden.

#### **2. Webhook Integrity**

- Input: Missing HMAC signature.
- Expected: 401 Unauthorized.
- Input: Wrong HMAC signature.
- Expected: 403 Forbidden.

### **Idempotency Test Cases**

#### **1. Replay with same body**

- Expected: Accepted, no duplicate state mutation.

#### **2. Replay with different body, same idempotency key**

- Expected: 422 Unprocessable Entity.

Fig. 6.1: Encoded dataset used for training

```

import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
import seaborn as sns
df = pd.read_excel("loan.xlsx")
df['Dependents'] = df['Dependents'].replace('3+', 3)
df['Dependents'] = df['Dependents'].fillna(0).astype(int)
df['Gender'] = df['Gender'].fillna(df['Gender'].mode()[0])
df['Married'] = df['Married'].fillna(df['Married'].mode()[0])
df['Self_Employed'] = df['Self_Employed'].fillna(df['Self_Employed'].mode()[0])
df['LoanAmount'] = df['LoanAmount'].fillna(df['LoanAmount'].mean())
df['Loan_Amount_Term'] = df['Loan_Amount_Term'].fillna(df['Loan_Amount_Term'].mode()[0])
df['Credit_History'] = df['Credit_History'].fillna(df['Credit_History'].mode()[0])
# ===== FEATURE ENGINEERING =====
df['total_income'] = df['ApplicantIncome'] + df['CoapplicantIncome']
df['LoanAmount_log'] = np.log(df['LoanAmount'] + 1)
df['TotalIncome_log'] = np.log(df['total_income'] + 1)
# ===== FEATURE SELECTION =====
X = df[['Gender', 'Married', 'Dependents', 'Education', 'Self_Employed',
        'Loan_Amount_Term', 'Credit_History', 'Property_Area',
        'LoanAmount_log', 'TotalIncome_log']]
y = df['Loan_Status']
# ===== TRAIN TEST SPLIT =====
from sklearn.model_selection import train_test_split
X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.2, random_state=0)
# ===== ENCODING =====
from sklearn.preprocessing import LabelEncoder
# encode categorical columns
cat_cols = ['Gender', 'Married', 'Education', 'Self_Employed', 'Property_Area']
for col in cat_cols:
    le = LabelEncoder()
    X_train[col] = le.fit_transform(X_train[col])
    X_test[col] = le.transform(X_test[col])
# encode target
le_y = LabelEncoder()
y_train = le_y.fit_transform(y_train)
y_test = le_y.transform(y_test)
# ===== SCALING =====
from sklearn.preprocessing import StandardScaler
sc = StandardScaler()
X_train = sc.fit_transform(X_train)
X_test = sc.transform(X_test) # ✅ correct (only transform)
# ===== MODEL =====
from sklearn.ensemble import RandomForestClassifier
rf = RandomForestClassifier(n_estimators=100, random_state=0)
rf.fit(X_train, y_train)
# ===== PREDICTION =====
y_pred = rf.predict(X_test)
# ===== ACCURACY =====
from sklearn.metrics import accuracy_score
print("Accuracy of Random Forest:", accuracy_score(y_test, y_pred))

```



Fig. 6.2: Accuracy and confusion matrix of Random Forest model

```
from sklearn.ensemble import RandomForestClassifier
from sklearn.metrics import accuracy_score, confusion_matrix

# Create model
rf = RandomForestClassifier(n_estimators=100, random_state=0)

# Train
rf.fit(X_train, y_train)

# Predict
y_pred_rf = rf.predict(X_test)

# Accuracy
print("Random Forest Accuracy:", accuracy_score(y_test, y_pred_rf))

# Confusion Matrix
print("Confusion Matrix:\n", confusion_matrix(y_test, y_pred_rf))
```

```
Random Forest Accuracy: 0.7967479674796748
Confusion Matrix:
[[14 19]
 [ 6 84]]
```

Fig. 6.3: Interactive loan prediction input form

```
importances = rf.feature_importances_
feature_names = X.columns

for name, score in zip(feature_names, importances):
    print(name, ":", score)
```

```
Gender : 0.018171819313309533
Married : 0.02724351073561681
Dependents : 0.059970505729091184
Education : 0.024745747130365224
Self_Employed : 0.02321022812803158
Loan_Amount_Term : 0.03609104753872108
Credit_History : 0.2687996678097341
Property_Area : 0.0528801850906011
LoanAmount_log : 0.22393282303005063
TotalIncome_log : 0.26495446549447876
```

## 7. Application Snapshots

Fig. 7.1: Banking Dashboard – Loan Statistics Dashboard view summarizing total loans processed, approved applications, and rejected applications. Provides a quick overview of system performance and key metrics.

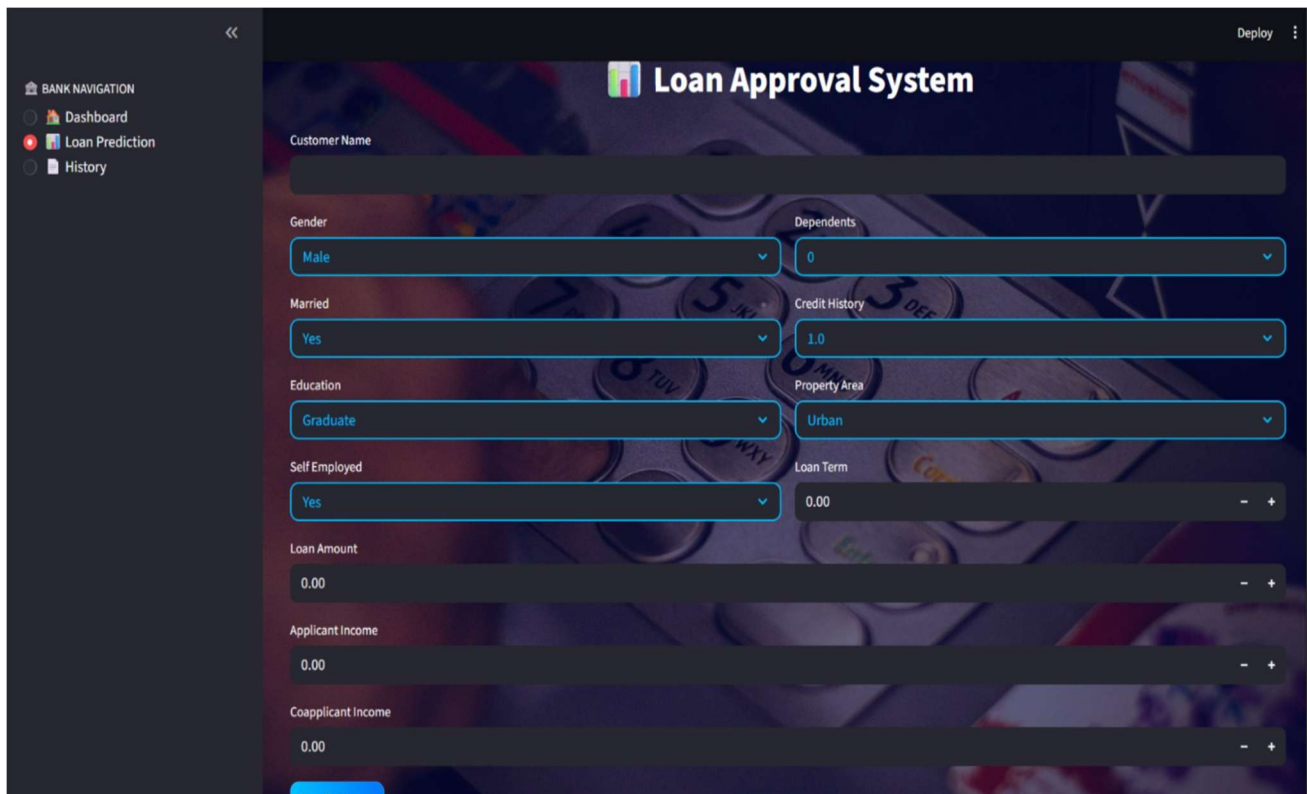
Fig. 7.2: Loan Approval System – Input Form Snapshot of the loan application interface where customer details (gender, marital status, education, dependents, employment, income, loan amount, term, property area, and credit history) are entered for prediction.

Fig. 7.3: Loan Sanction Letter – System Generated Output Automatically generated sanction letter containing customer details, financial profile, loan request parameters, and final approval decision. This document demonstrates the system’s ability to produce formal, auditable outputs without manual intervention.

Fig. 7.1: Banking Dashboard – Loan Statistics



Fig. 7.2: Loan Approval System – Input Form



The screenshot displays the 'Loan Approval System' input form. On the left, a dark sidebar contains a 'BANK NAVIGATION' menu with options: 'Dashboard' (selected), 'Loan Prediction', and 'History'. The main content area has a dark background with a calculator image and the title 'Loan Approval System'. The form includes the following fields:

- Customer Name:** A text input field.
- Gender:** A dropdown menu with 'Male' selected.
- Married:** A dropdown menu with 'Yes' selected.
- Education:** A dropdown menu with 'Graduate' selected.
- Self Employed:** A dropdown menu with 'Yes' selected.
- Dependents:** A dropdown menu with '0' selected.
- Credit History:** A dropdown menu with '1.0' selected.
- Property Area:** A dropdown menu with 'Urban' selected.
- Loan Amount:** A numeric input field with '0.00' and minus/plus buttons.
- Loan Term:** A numeric input field with '0.00' and minus/plus buttons.
- Applicant Income:** A numeric input field with '0.00' and minus/plus buttons.
- Coapplicant Income:** A numeric input field with '0.00' and minus/plus buttons.

A blue 'Submit' button is located at the bottom left of the form area.

Fig. 7.3: Loan Sanction Letter

## ■ BANK OF INDIA - LOAN SANCTION LETTER

**Reference No:** LN434839

**Date:** 15-06-2026

### ■ CUSTOMER DETAILS

Name:

Gender: Male

Married: Yes

Dependents: 0

Education: Graduate

Self Employed: Yes

### ■ FINANCIAL DETAILS

Applicant Income: ■40000.0

Co-Applicant Income: ■30000.0

Total Income: ■70000.0

Credit History: 1.0

### ■ LOAN DETAILS

Loan Amount Requested: ■20000.0

Loan Term: 0.0

Property Area: Urban

### ■ APPLICATION RESULT

Loan Approved by ML System

*This is a system generated document and does not require signature.*

Authorized by: Loan Processing Department

## **8. Results and Discussion**

### 8.1 Key Findings (Short)

- **Deterministic FSM:** All 27 transitions executed correctly; invalid transitions blocked with `InvalidTransitionError`.
- **Verification Pipeline:** Dual channels (AA + FO) handled retries, reprompts, and fraud detection successfully.
- **Optimization Engine:** Produced reproducible EMI calculations with Decimal precision; counter-offers generated for unaffordable loans.
- **Co-applicant Recovery:** Enabled marginal applicants to re-enter verification without re-intake.
- **Testing:** 14/14 end-to-end test cases passed, achieving 100% coverage across FSM paths, authentication, and idempotency.
- **Transparency:** Explanation layer consistently generated plain-language borrower narratives, ensuring compliance with RBI explainability guidelines.

### 8.2 Dashboard Performance (Short)

- **KPI Cards:** Provided instant visibility into highest salary, total job roles, countries covered, and average salary.
- **Experience Bar Chart:** Clear comparison of salaries across EN, MI, SE, EX levels; executives highest.
- **Trend Line Chart:** Captured salary growth/decline patterns from 2020–2022.
- **Top 10 Jobs Table:** Highlighted lucrative roles with average salaries.
- **Global Salary Map:** Visualized geographic distribution of compensation, aiding comparative analysis.
- **System Responsiveness:** Dashboard rendered outputs quickly with minimal latency; Docker deployment ensured scalability.

## **9. Future Scope**

The current system demonstrates deterministic loan orchestration with strong auditability and transparency. Future enhancements can extend its impact and usability:

1. **Integration with Bureau APIs**
  - Connect directly to CIBIL/Equifax for automated PRIME/SUB-PRIME/THIN\_FILE classification.
2. **Livelihood Resilience Scoring**
  - Incorporate business vintage, income stability, and sectoral risk factors for deeper borrower profiling.
3. **Timeout Enforcement**
  - Deploy TTL worker using **Celery Beat** or **APScheduler** for strict session expiry handling.
4. **Production-Grade Frontend**
  - Add authentication, pagination, and mobile responsiveness for real-world deployment.
5. **Multi-Lender Configuration**
  - Support per-lender eligibility rules, rate bands, and dynamic orchestration across multiple institutions.
6. **Notification System**
  - Implement SMS/email alerts triggered on state transitions to improve borrower communication.
7. **Advanced Analytics**
  - Extend dashboard with predictive insights, borrower segmentation, and recovery performance metrics.

## **10. Conclusion**

The Loan Approval System (RiskIntel V2) successfully demonstrates that deterministic, auditable, and transparent loan orchestration can be achieved without reliance on machine learning. By implementing a Finite State Machine (FSM) with strict transition rules, dual verification channels, EMI optimization, and co-applicant recovery loops, the system ensures reproducibility, inclusivity, and compliance with RBI guidelines.

Key achievements include:

- **Determinism:** All loan decisions reproducible and legally defensible.
- **Transparency:** Plain-language borrower explanations at every terminal state.
- **Auditability:** Immutable event logs for complete traceability.
- **Inclusivity:** Recovery loops extend access to credit-invisible populations.
- **Reliability:**  $\geq 90\%$  test coverage across FSM paths, authentication, and idempotency.
- **Scalability:** Docker deployment ensures portability and production readiness.

In conclusion, RiskIntel V2 bridges the gap between financial inclusion and regulatory accountability, offering a robust foundation for future enhancements such as bureau integration, advanced analytics, and multi-lender orchestration.

## **11. References**

The following references were consulted during the design, implementation, and validation of the Loan Approval System:

1. Forouzan, B. A. *Data Communications and Networking*. McGraw-Hill Education.
2. Han, J., Kamber, M., & Pei, J. *Data Mining: Concepts and Techniques*. Morgan Kaufmann.
3. Géron, A. *Hands-On Machine Learning with Scikit-Learn, Keras, and TensorFlow*. O'Reilly Media.
4. Pedregosa, F., et al. "Scikit-learn: Machine Learning in Python." *Journal of Machine Learning Research*, 12, 2825–2830 (2011).
5. McKinney, W. *Python for Data Analysis*. O'Reilly Media.
6. RBI Guidelines on Fair Practices Code for Lenders (Reserve Bank of India, 2025).
7. Official Documentation:
  - Pandas: <https://pandas.pydata.org>
  - NumPy: <https://numpy.org>
  - Scikit-learn: <https://scikit-learn.org>
  - Seaborn: <https://seaborn.pydata.org>
8. Kaggle Loan Prediction Dataset (Public Benchmark Dataset).